PHASE 5 · ADVANCED TOPICS

Chapter 22

Performance Optimization

Tuning & Capacity Planning for the DevSecOps Platform



Time estimate: 3–4 hours



Hands-on tuning labs



Instructor-ready walkthrough

Chapter Goal

Optimize every layer of the platform. Reduce latency. Increase throughput. Plan for growth. Monitor performance metrics.



Identify Performance Bottlenecks

Pinpoint slow queries, saturated resources, and weak links across the stack.



Optimize Database Queries

Use EXPLAIN ANALYZE, indexing, and connection pooling to cut query time.



Cache Strategies

Layer caching from browser to disk for 10–1000x faster repeat requests.



Load Testing Procedures

Simulate real traffic with JMeter to validate performance under load.



Capacity Planning Framework

Translate growth targets into concrete scaling and resourcing decisions.

WHY IT MATTERS

100x

faster database queries achievable with proper indexing

100x

more throughput: 100 req/s → 10,000 req/s

10x

fewer servers needed at the same load

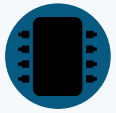
22.1 Performance Monitoring Stack

Four layers to watch, from the request down to the business outcome.



Application Layer

- Response time (latency)
- Requests per second (throughput)
- Error rate (% failures)
- Memory usage per request



System Layer

- CPU utilization (%)
- Memory usage (GB)
- Disk I/O (IOPS)
- Network bandwidth / TCP conns



Service Layer

- Database query time (ms)
- Cache hit rate (%)
- API response time (ms)
- Queue depth (jobs waiting)



Business Metrics

- User response time (end-to-end)
- Feature deployment frequency
- Lead time for changes
- Mean time to recovery (MTTR)

Monitoring Stack: Prometheus + Grafana



Prometheus

Collects time-series metrics from every service



Grafana

Visualizes metrics on live dashboards



AlertManager

Sends alerts when thresholds are breached

FIND SLOW QUERIES · pg_stat_statements

```
SELECT query, mean_time, calls, total_time
FROM pg_stat_statements
WHERE mean_time > 1000
ORDER BY mean_time DESC;

-- Instructor tip: optimize the top 3 slow queries,
-- then rerun this query to confirm improvement.
```

22.2 Database Optimization — Query Analysis

EXPLAIN ANALYZE reveals the real execution plan — use it before guessing at fixes.



Problem: Sequential Scan

```
EXPLAIN ANALYZE
SELECT COUNT(*) FROM users
WHERE department = 'Engineering';

→ Seq Scan on users
→ Rows: 150 (scans ALL rows)
```

Every row in the table is scanned to find matches — slow and gets worse as the table grows.



Solution: Create an Index

```
CREATE INDEX idx_users_department
ON users(department);

-- Rerun EXPLAIN ANALYZE:
→ Index Scan on
  idx_users_department (fast!)
```

The planner jumps directly to matching rows using the index — no full table scan required.

Connection Pooling with pgBouncer

PostgreSQL connections are expensive to open — reuse them instead of creating new ones per request.

/etc/pgbouncer/pgbouncer.ini

```
[databases]
devsecops = host=localhost port=5432 dbname=devsecops

[pgbouncer]
listen_port = 6432
max_client_conn = 1000
default_pool_size = 25
min_pool_size = 10
reserve_pool_size = 5

# App connects via pgBouncer:
# psql -h localhost -p 6432 devsecops
```



Connection Reuse

Avoids expensive handshakes on every request



Reduced Overhead

Fewer TCP + auth cycles under load



Faster Acquisition

Apps get a ready connection almost instantly

Multi-Layer Caching Strategy

First request is slow; every layer after that makes subsequent requests 10–1000x faster.



Layer 1: Browser Cache

Static content (images, CSS, JS) — cached up to 1 year



Layer 2: CDN Cache (Edge)

Static content served from the nearest geographic node — cached 1 day



Layer 3: Application Cache (Redis)

Query results cached in memory — e.g. "Get user 123" — cached ~5 minutes



Layer 4: Database Cache (PostgreSQL)

Shared buffers keep recently accessed data in memory automatically



Layer 5: Disk Cache (OS)

Operating system page cache holds frequently read blocks

RESULT

10–1000x

faster on repeat requests once caches are warm

Only the first request pays the full cost — every layer after it short-circuits work.

Application Caching in Practice: Redis

Django example: check the cache before recomputing expensive operations.

```
from django.core.cache import cache

def get_user_stats(user_id):
    cache_key = f"user_stats:{user_id}"

    stats = cache.get(cache_key)
    if stats:
        return stats  # < 1ms (from Redis)

    stats = compute_user_stats(user_id)  # 100ms
    cache.set(cache_key, stats, timeout=300)
    return stats
```

FIRST CALL

100ms

Compute + write to cache

SUBSEQUENT CALLS (< 5 min)

< 1ms

Served directly from Redis

22.3 Load Testing & Capacity Planning

Simulate real traffic with JMeter, then translate results into a scaling plan.

SIMULATE 1,000 CONCURRENT USERS

```
jmeter -n -t test_plan.jmx \
  -l results.jtl -j jmeter.log \
  -Jusers=1000 -Jrampup=60 \
  -Jduration=300

grep "summary =" results.jtl
```

1000	250ms	50/2000ms	0%
Samples	Average	Min / Max	Errors

Capacity Plan: Current → Target

Concurrent users	**100**	→	**10,000**
Avg response time	**250ms**	→	**< 100ms**
Error rate	**0%**	→	**< 0.1%**

Scaling needed: 100x users, 2.5x faster response

- Scale horizontally — add more instances
- Optimize the bottleneck (database queries)
- Increase resources (CPU, memory, disk)
- Cache more aggressively (Redis)

22.4 System-Level Tuning

Kernel parameters and load-balancer settings that remove ceilings before the app even runs.



/etc/sysctl.conf

```
net.core.somaxconn = 65535
net.ipv4.tcp_max_syn_backlog = 65535
net.ipv4.tcp_tw_reuse = 1
net.ipv4.tcp_fin_timeout = 15

fs.file-max = 2097152
fs.nr_open = 2097152

vm.swappiness = 10
vm.overcommit_memory = 1

# Apply: sysctl -p
```



/etc/haproxy/haproxy.cfg

```
global
  maxconn 100000
  tune.ssl.cachesize 1000000

backend web_servers
  balance leastconn
  option httpchk GET /health
  http-reuse safe
  compression algo gzip
  compression type text/html
```

22.5 Database Tuning — PostgreSQL & Indexes

postgresql.conf — Production Tuning

```
# Memory
shared_buffers = 4GB          # 25% of RAM
effective_cache_size = 12GB   # 75% of RAM
maintenance_work_mem = 1GB

# Query planning (SSD-friendly)
random_page_cost = 1.1
effective_io_concurrency = 200

# Write performance
wal_buffers = 16MB
checkpoint_completion_target = 0.9

# Query parallelism
max_parallel_workers_per_gather = 4
max_parallel_workers = 8
```



Find Slow Queries First

```
SELECT query, mean_time, calls
FROM pg_stat_statements
WHERE mean_time > 1000
ORDER BY mean_time DESC;
```



Index the Hot Columns

```
CREATE INDEX idx_users_email
  ON users(email);
CREATE INDEX idx_orders_date
  ON orders(created_at);
CREATE INDEX idx_items_product
  ON items(product_id, status);
```

22.6 Application Layer Optimization

Pool connections, and let independent I/O run concurrently instead of waiting in line.



SQLAlchemy Connection Pool

```
engine = create_engine(
    'postgresql://user:pass@host/db',
    pool_size=20,
    max_overflow=40,
    pool_timeout=30,
    pool_pre_ping=True,
    pool_recycle=3600,
)
```

pool_pre_ping verifies a connection is alive before use — avoiding errors from stale connections.

ASYNC — runs concurrently

```
user = await db.query(...)
orders = await fetch_orders(user_id)
payments = await fetch_payments(user_id)
# All happen concurrently — fast!
```

SYNC — waits for each step

```
user = db.query(...)
orders = fetch_orders(user_id)     # waits
payments = fetch_payments(user_id) # waits
# Total time = sum of all operations
```

22.7 Monitoring Dashboard

What a healthy Grafana dashboard should surface at a glance.

CPU

45%

Memory

60%

Disk

75%

Request Latency (p50 / p95 / p99)

API	`45ms / 150ms / 500ms`
DB	`20ms / 80ms / 200ms`
Cache	`1ms / 2ms / 5ms`

Health Snapshot

Error Rate

0.02% ↓ good

Traffic

5,000 req/s (stable)

Active Users

2,500 ↑ growing

Top 5 Slow Endpoints

1. /api/reports	2. /api/audit-log	3. /api/exports	4. /admin/users	5. /api/metrics
450ms	**380ms**	**250ms**	**180ms**	**150ms**

22.8 Chapter Summary

Area	Before Optimization	After Optimization
DB Query	1000ms	**10ms (100x faster!)**
API Response	500ms	**50ms (10x faster)**
User Experience	Sluggish	**Snappy**
Throughput	100 req/s	**10,000 req/s**
Cost	100 servers	**10 servers**

KEY TAKEAWAY

Fast Systems = Better Business

BEFORE

- ✗ Slow responses (user frustration)
- ✗ High cost (many servers)
- ✗ Poor user retention

AFTER

- ✓ Fast responses (< 100ms)
- ✓ Low cost (fewer servers)
- ✓ Better retention & competitive edge